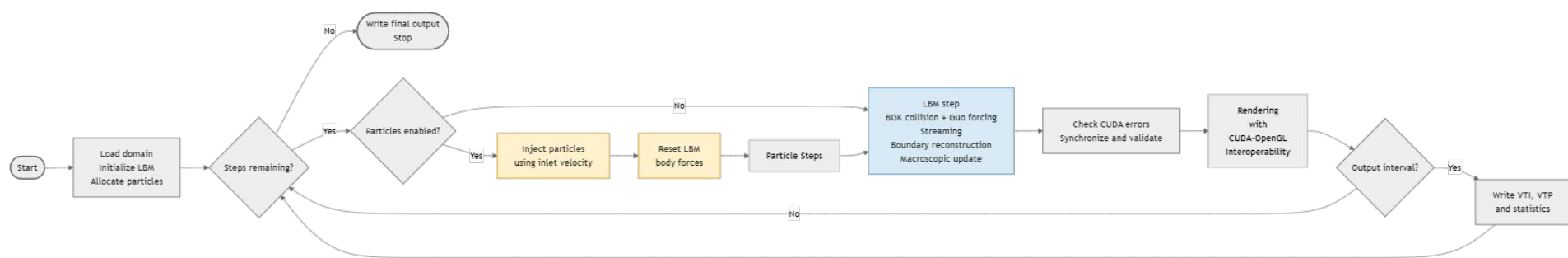




Blood Vessel Simulation

HESP Project

Project Overview





Core Simulation Concepts

Fluid -> LBM

Particles -> Spring

Interaction -> Particle <> Fluid

Mesh -> Generating

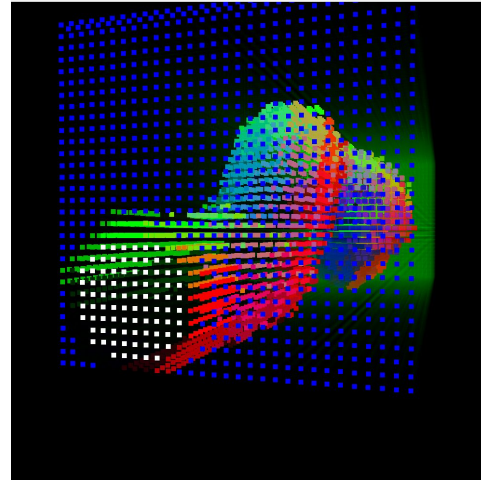
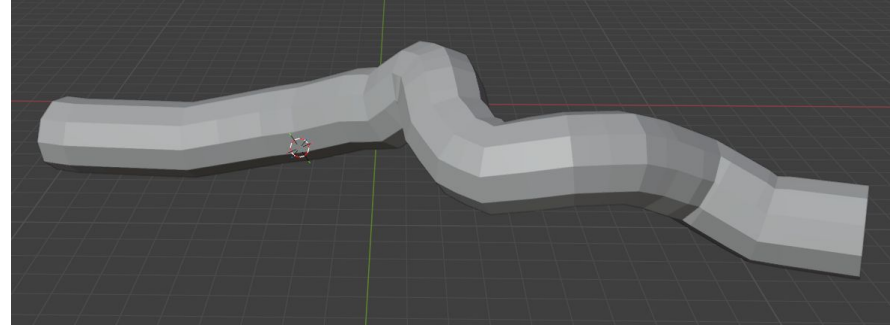
Mesh

Geometry Vessel

An Open Triangle Mesh is created with Blender, and exported as raw triangle list to the Voxelizer.

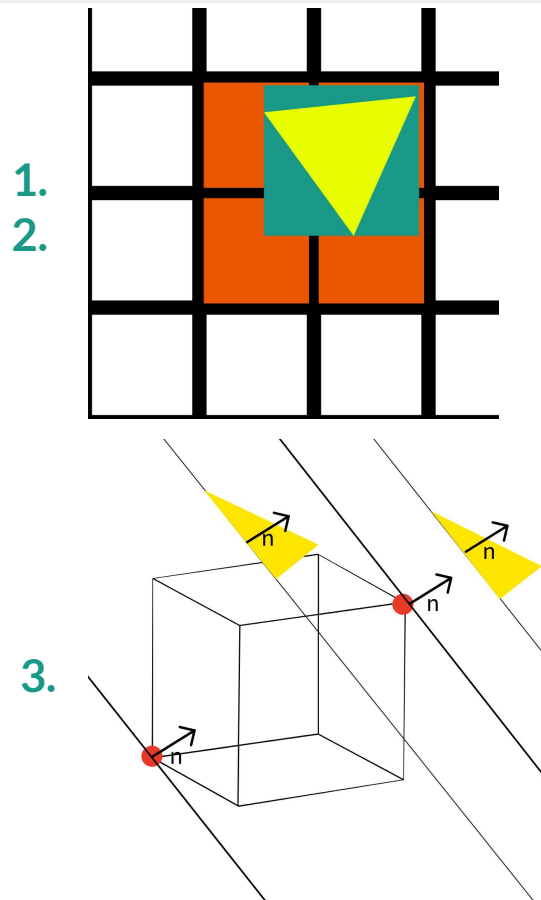
Triangle Voxelizer

- A Triangle marks the grid cells it overlaps as Solid voxels, and assigns the triangle normal to the voxel.
- Scans the voxels at the start and end domains, to find Inlet and Outlet cells.



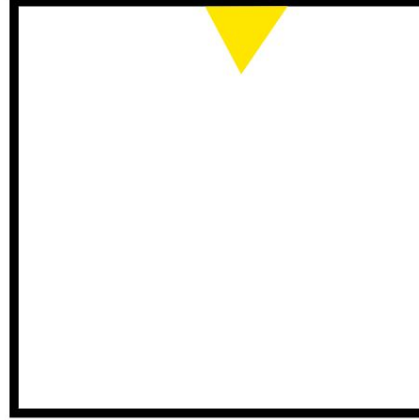
Voxelization Steps

1. Compute bounding box of Triangle T.
2. Check if the bounding box overlaps the grid cell.
3. Check if the Triangle's plane overlaps the grid cell using the plane formed at the cell's two critical points.
4. For each of the three coordinate planes (xy, xz, yz), check if the 2D projections of Triangle and grid cell into these planes overlap (All 3 plane test must show intersection to mark the grid cell as Solid).

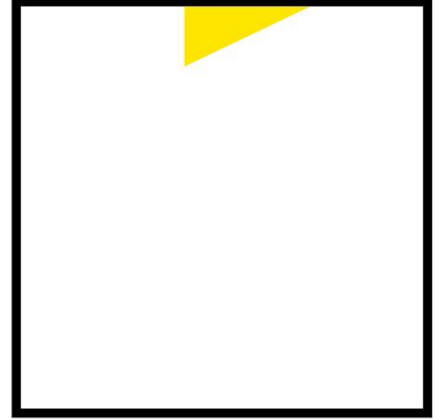


4.

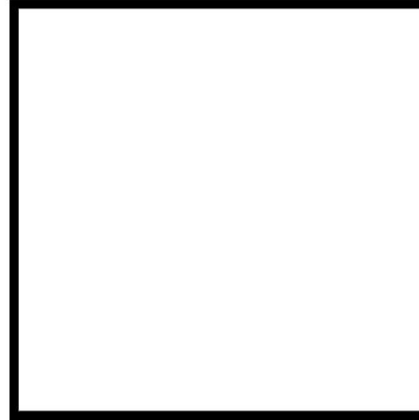
xy-plane



yz-plane



zx-plane





Collision

Collision Operator

During the collision every population is relaxed toward the equilibrium distribution.

$$f_q^* = f_q - \omega(f_q - f_q^{eq}) + F_q$$

Equilibrium Distribution

The local equilibrium state modeled to recover Navier-Stokes macroscopic behavior.

$$f_q^{eq} = w_q \rho \left[1 + 3 \mathbf{c}_q \cdot \mathbf{u} + \frac{9}{2} (\mathbf{c}_q \cdot \mathbf{u})^2 - \frac{3}{2} \mathbf{u} \cdot \mathbf{u} \right]$$

Guo Forcing Scheme

We apply the effect of the external forces to the fluid through the Guo forcing.

$$F_q = w_q \left(1 - \frac{\omega}{2} \right) [3(\mathbf{c}_q - \mathbf{u}) + 9(\mathbf{c}_q \cdot \mathbf{u})\mathbf{c}_q] \cdot \mathbf{F}$$



Stream

Streaming Operator

After the collision the populations propagate to the neighbour cells. We use a pull approach, every cell reads populations coming from the source cell.

$$f_q(\mathbf{x}, t + 1) = f_q^*(\mathbf{x} - \mathbf{c}_q, t)$$

Bounce-back

If the neighbour cell is Solid we apply the bounce-back. We take the populations that after the collision are going to the wall cell and we copy them in the opposite direction.

$$f_q(\mathbf{x}, t + 1) = f_{\bar{q}}^*(\mathbf{x}, t)$$



Inlet

Density Operator

The density is not imposed and it is calculated from the known populations where S_0 are the populations with null normal component and S_- are the populations that move outside the domain.

$$\rho_{in} = \frac{S_0 + 2S_-}{1 - u_{z,in}}$$

Missing pop. computation

The missing populations are reconstructed assuming the non equilibrium part is equal to the one in the opposite direction.

$$f_q = f_q^{eq} + (f_{\bar{q}} - f_{\bar{q}}^{eq})$$



Outlet

Imposed Density

We impose density equal to one. Our fluid is not compressible so the pressure should be unitary everywhere. The imposed density back-propagates in the fluid

$$\rho_{out} = 1$$

Normal velocity operator

The normal velocity is not imposed and it is calculated from the known populations where S_0 are the populations with null normal component and S_+ are the populations that move outside the domain.

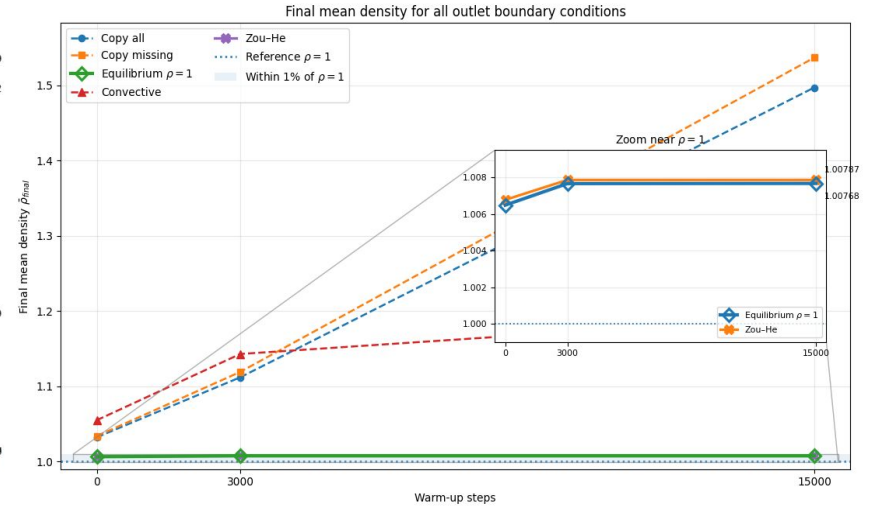
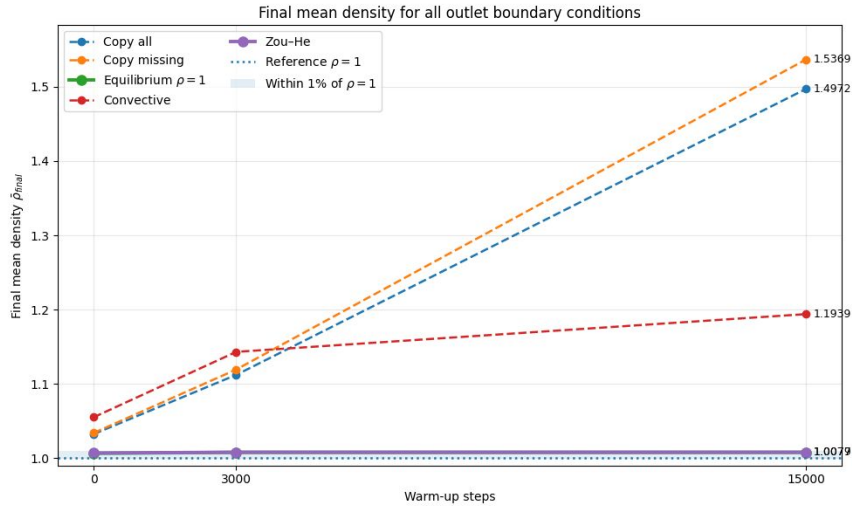
$$u_{z,out} = \frac{S_0 + 2S_+}{\rho_{out}} - 1$$

Missing pop. computation

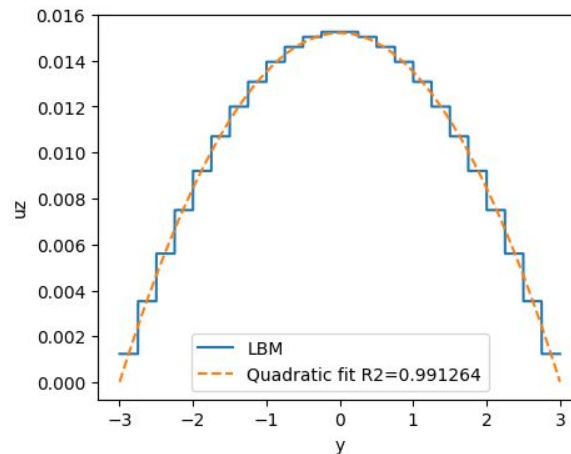
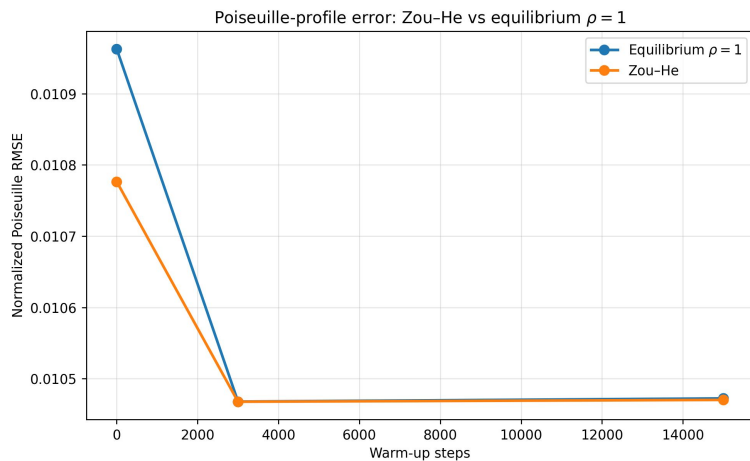
The missing populations are reconstructed with the equilibrium relation using the imposed density.

$$f_q = f_q^{eq}(\rho_{out}, \mathbf{u}_{out}) + f_{\bar{q}} - f_{\bar{q}}^{eq}(\rho_{out}, \mathbf{u}_{out})$$

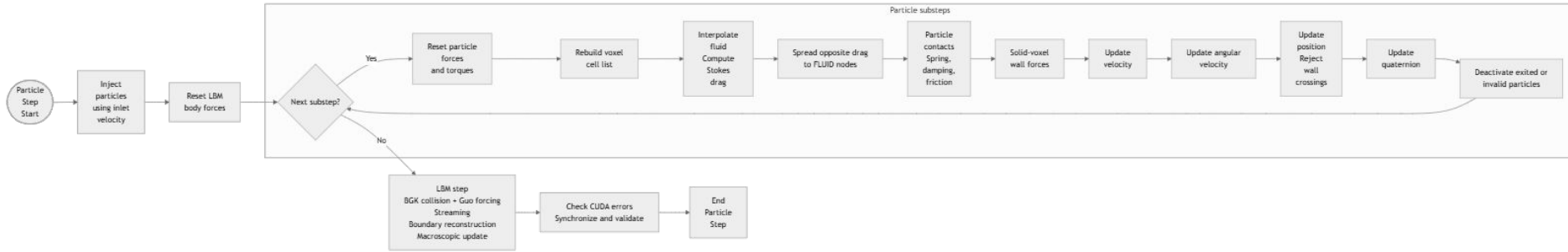
Tried outlet models results



Zou-He vs Equilibrium_rho1 and Poiseuille Like Fluid Behaviour



Particles



$$F_i = F_{drag,i} + \sum_j F_{contact,ij} + F_{wall,i}$$



Particles (Particle - Particle)

Overlapping Particles if (searched in own voxel and 26 neighbor cells, $3 * 3 * 3$ area):

$$\delta = r_i + r_j - \|\mathbf{x}_i - \mathbf{x}_j\| > 0$$

Linear Spring-Dashpot Force: (n is normal vector pointing from particle j to particle i) (equal and opposite applied to both particles):

$$F_n = \max(0, k_c \delta - c_c v_n)$$

The trial tangential force (viscous):

$$\mathbf{F}_t^* = -c_c \mathbf{v}_t$$

Coloumb Bound:

$$\|\mathbf{F}_t\| \leq \mu_f F_n$$



Particles

- Three species: red blood cells, platelets, and leukocytes
- Particles are rigid bodies stored and processed on the GPU
- RBCs have ellipsoidal orientation data, but all drag and contact calculations use an effective spherical radius
- Particles are injected at inlet cells using the local fluid velocity
- Particles are deactivated after leaving the grid or reaching an outlet
- linked-cell list



Particles

1. Inject the requested species at inlet cells using the current fluid velocity.
2. Reset the LBM body-force arrays once.
3. For every particle substep:
 1. reset particle forces, torques, and contact counts;
 2. reset and rebuild the linked-cell list;
 3. interpolate the unchanged current LBM fields and compute drag/reaction;
 4. compute particle contacts and wall forces;
 5. cap total force and update linear velocity;
 6. cap speed and update angular velocity;
 7. update position, check/cap velocity, and update orientation;
 8. deactivate particles outside the grid or in an outlet cell;
 9. copy the current force into ``F_old``.
4. Advance the LBM once: BGK collision with forcing, streaming, boundary conditions, and macroscopic reconstruction.



Particles (Fluid Drag)

Tau is BGK relaxation time, r is the spherical radius of the particle, u_f is interpolated fluid velocity and v_p is particle velocity

$$\mu = \frac{\rho_f \times (\tau - 0.5)}{3}$$

$$F_{drag} = 6 \times \pi \times \mu \times r \times (u_f - v_p)$$

The equal and opposite force is distributed back to the same fluid cells

Because of N particle substeps, each substep only applies force with factor $1/N$



Particles (Particle - Wall)

Penalty Force (for each neighboring solid face)

$$\delta_w = r - d_w$$

$$F_{wall} = \max(0, k_w \times \delta_w - c_w \times v_n) \times n_{wall}$$



Particles (Integrator)

```
v <- v + 0.5 * (F_old + F_new) / m * dt_p
```

```
x <- x + v * dt_p + 0.5 * F_new / m * dt_p^2
```

```
F_old <- F_new
```



Particles (Rotation)

Angular Velocity (explicit Euler):

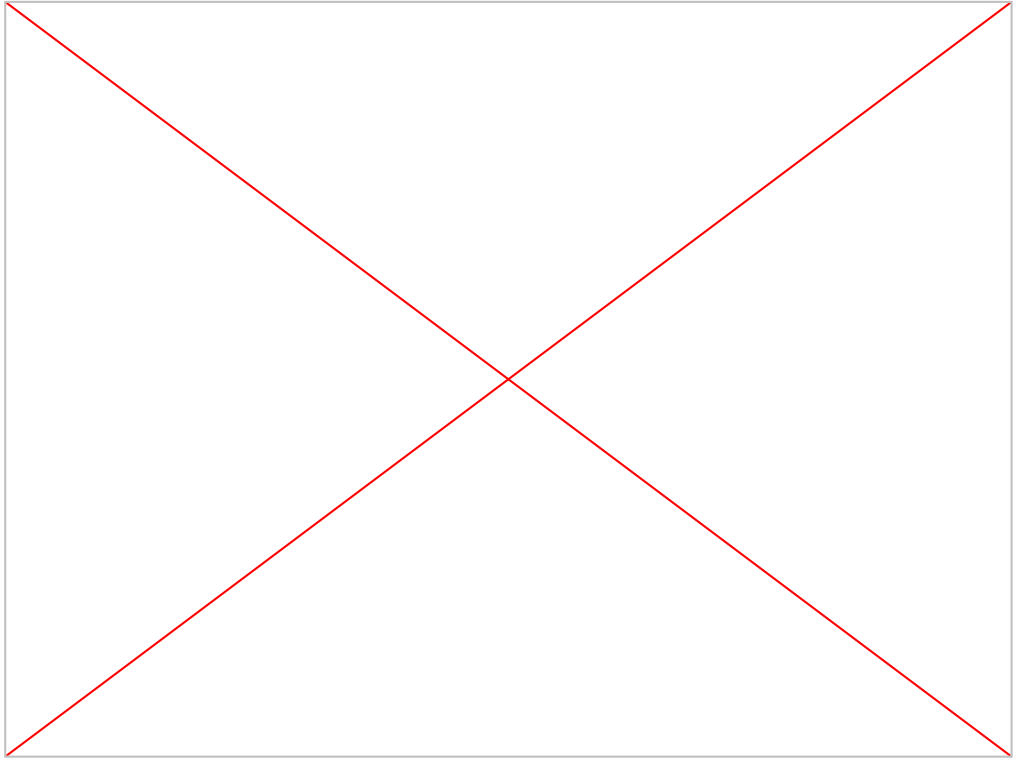
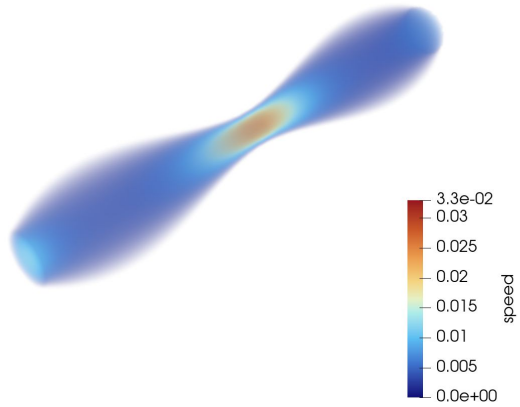
$$(\boldsymbol{\omega} \leftarrow \boldsymbol{\omega} + \mathbf{I}^{-1} \boldsymbol{\tau} \Delta t_p)$$

Rotation Update:

$$\begin{aligned} \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes (0, \boldsymbol{\omega}) \\ \mathbf{q} &\leftarrow \text{normalize} (\mathbf{q} + \dot{\mathbf{q}} \Delta t_p) \end{aligned}$$



Results





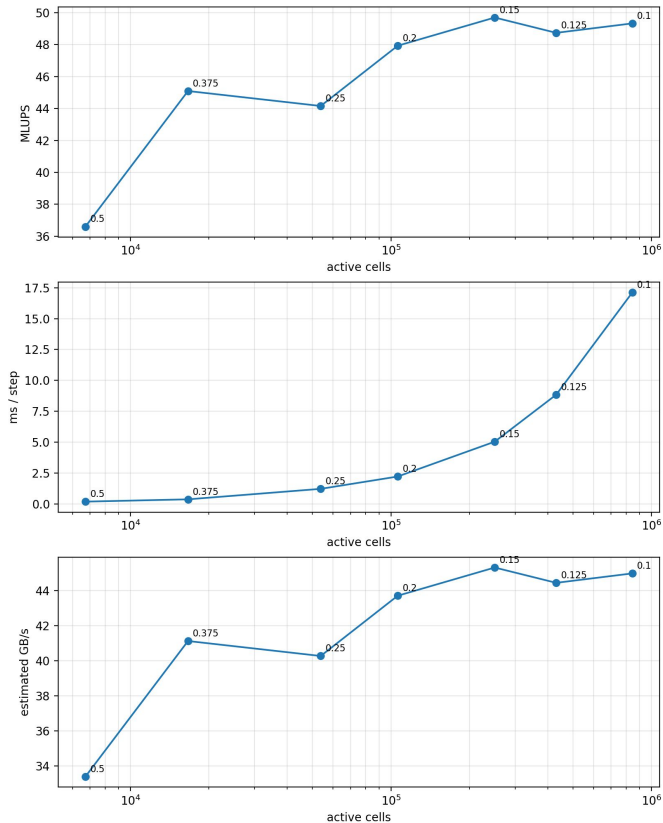
Mesh preprocessing using gmsh and trimesh

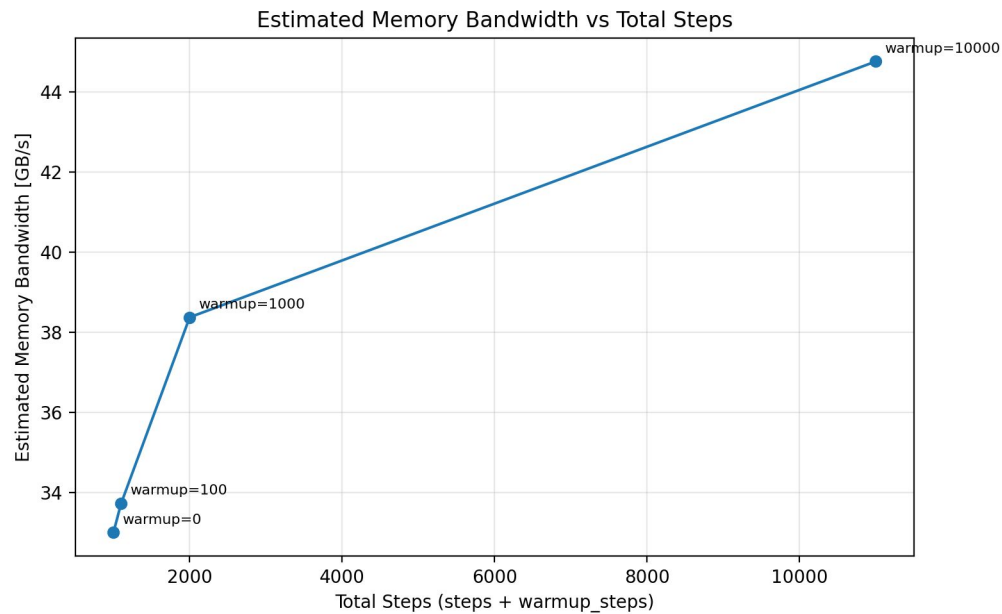
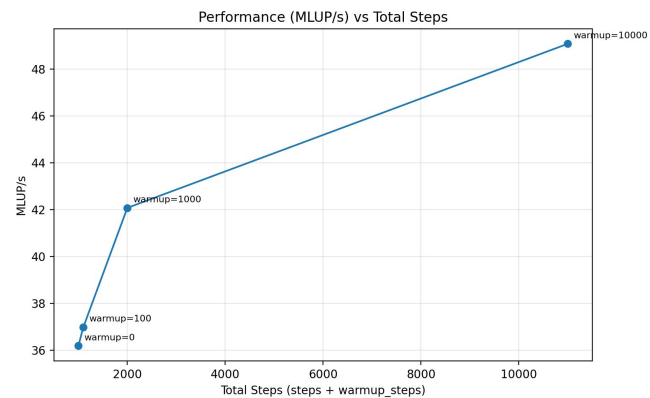
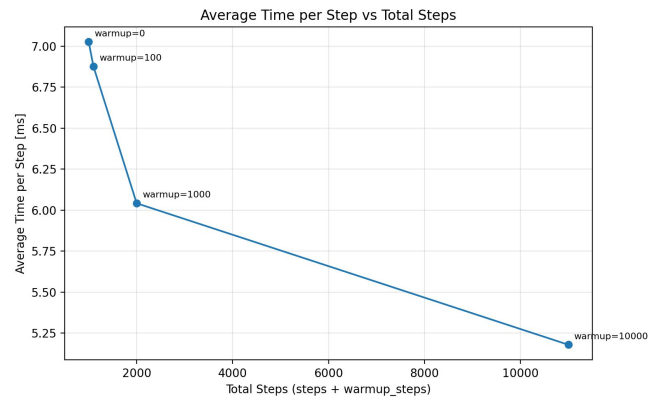
We have also another alternative for the preprocessing through already implemented python libraries that take a file .geo and build a voxelized mesh ready to be used, Gmsh is used during preprocessing to generate the surface and volumetric mesh and to identify the wall, inlet, and outlet physical regions. Trimesh is then used to vectorize the voxelization process by testing batches of cell centers against the geometry. The meshes shown in the previous slide were generated using this preprocessing pipeline.

Profiling

As the mesh is refined (smaller dx), the number of active cells increases, leading to a longer execution time per iteration. However, the throughput (MLUPS) also improves, reaching a plateau around **49–50 MLUPS** for the finest meshes. The estimated memory bandwidth follows the same trend, saturating at approximately **45 GB/s**, indicating that the implementation becomes **memory-bandwidth limited** rather than compute-limited for large problem sizes. The slight fluctuations at the highest resolutions are likely due to measurement noise and cache effects rather than fundamental performance changes.

The profiling has been done on a cip.pool.fau.de computer mounting a Nvidia Geforce RTX 2080





Plots of performance vs Step number



Other



Output / Visualization



CUDA Features used